

# Reviewer Pack — Minimal Monodromy Group Order and Communication Complexity R...

Entry 82933da8aa2a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 05:46:52 UTC

## Minimal Monodromy Group Order and Communication Complexity Rank Variance

**Entry ID:** 82933da8aa2a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 05:46:52 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Monodromy Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

#### Statement:

For every Boolean function  $f: \{0,1\}^n \rightarrow \{0,1\}$ , the order of its monodromy group  $|G_f|$  is linearly correlated with the variance of its communication complexity rank, such that  $|G_f| = \Theta(\text{CommRankVar}(f))$ .

#### Rationale (proposer's reasoning):

Monodromy theory studies the symmetries and automorphisms in algebraic varieties, which could potentially reveal hidden structures in Boolean functions that influence their communication complexity. A linear correlation would suggest a direct connection between these properties.

**Taxonomy category:** AlgebraicGeometry × CommunicationComplexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d27abc9926c977e5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For Boolean functions  $f: \{0,1\}^n \rightarrow \{0,1\}$ ,  $|G_f|$  is considered supported if  $\text{support\_fraction} \geq 0.8$  AND  $\text{metric\_mean} \leq 3$ , where  $\text{support\_fraction}$  is the proportion of  $n \leq 40$  functions with  $|G_f| = \Theta(\text{CommRankVar}(f))$ , and  $\text{metric\_mean}$  is the mean value of  $|G_f|$  for these functions.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | UNCERTAIN        | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "Minimal Monodromy Group Order" AND "Communication Complexity Rank Variance" - "Algebraic Geometry" AND "Matrix Rank Variance" AND "Monodromy Theory" - "Boolean function" AND "monodromy group order" AND "communication complexity"

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=240.4s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank_variance(f):
        n = int(math.log2(len(f)))
        rank = [f.count(i) for i in set(f)]
        mean = sum(rank) / len(rank)
        variance = sum((x - mean)**2 for x in rank) / len(rank)
        return variance

    def monodromy_group_order(f):
        # Placeholder function to simulate computing the monodromy group order
        n = int(math.log2(len(f)))
        return 2**n

    metric_name = "monodromy_group_order"
    instances_tested = 0
    n_max = 0
    total_metric_value = 0.0
    support_count = 0
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n

        for _ in range(5):
```

```

    f = generate_boolean_function(n)
    instances_tested += 1
    G_f = monodromy_group_order(f)
    CommRankVar_f = communication_complexity_rank_variance(f)

    total_metric_value += G_f

    if abs(G_f - CommRankVar_f) <= 0.1 * max(G_f, CommRankVar_f):
        support_count += 1

metric_mean = total_metric_value / instances_tested
support_fraction = support_count / instances_tested

conjecture_holds = support_fraction >= 0.8 and metric_mean <= 3

return {
    "metric_name": metric_name,
    "metric_value": metric_mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std={math.sqrt(sum((v - sum(metric_values)/len(metric_values))**2 for v in metric_values)/len(metric_values)):.2f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std={math.sqrt(sum((v - sum(metric_values)/len(metric_values))**2 for v in metric_values)/len(metric_values)):.2f}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

Critic verdict: CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

**9. Final verdict & safety rail**

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out before producing data, which means it did not meet the pre-registered support conditions for |G\_f| to be considered supported. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the allowed time frame.

**11. Audit log (LLM calls)**

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15831        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10764        |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8655         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 11404        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 24583        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9755         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8832         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9562         |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 39532        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138919 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/82933da8aa2a.jsonl`)

**12. Reproducibility**

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/82933da8aa2a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/82933da8aa2a.tar.gz` (if generated)